Revisión práctica de arquitectura de flujos: APIs, fallbacks y logging.

Este módulo ofrece un análisis detallado y aplicado de arquitecturas centradas en APIs y webhooks para la orquestación de flujos en entornos de healthcare y biotecnología. Se abordan estrategias prácticas para diseñar rutas de fallo (fallbacks) y políticas de espera que aseguren la robustez y continuidad del servicio. Además, se introduce el uso de logs, métricas y observabilidad para el monitoreo en tiempo real y diagnóstico efectivo, preparando al profesional para construir sistemas escalables, mantenibles y confiables en producción.

Introducción

Imagina que estás diseñando un sistema automatizado para gestionar la administración de medicamentos en un hospital. Este sistema debe integrar múltiples servicios: bases de datos de pacientes, sistemas de inventario, y alertas en tiempo real para el personal médico. ¿Cómo asegurar que esta orquestación funcione sin interrupciones, incluso cuando alguno de los servicios falle o responda lentamente? En este contexto, dominar el diseño de arquitecturas basadas en APIs REST y webhooks, junto con estrategias robustas de fallback y un sistema efectivo de logging y observabilidad, es fundamental para garantizar la continuidad y confiabilidad del flujo.

En esta unidad, exploraremos cómo construir arquitecturas de flujos que integren estos elementos clave, con ejemplos concretos del sector healthcare y biotecnología. Al finalizar, podrás diagramar sistemas que no solo funcionen bajo condiciones ideales, sino que también manejen fallos y permitan un monitoreo en tiempo real para una rápida detección y resolución de problemas.

Fundamentos Teóricos

Métodos HTTP (GET, POST, DELETE, etc.)

Son **verbos** que indican qué quiere hacer el cliente con el recurso.

GET

Sirve para **obtener información**. No debería modificar nada en el servidor.

Ejemplo: pedir la lista de productos.

POST

Sirve para **crear un recurso nuevo** o ejecutar una acción que cambia estado.

Ejemplo: crear un usuario o enviar un formulario.

PUT

Se usa para **reemplazar completamente** un recurso existente.

Ejemplo: actualizar todos los datos de un perfil.

PATCH

Se usa para **modificar parcialmente** un recurso.

Ejemplo: cambiar solo el email del usuario.

DELETE

Sirve para **eliminar** un recurso.

Ejemplo: borrar una orden.

> *Idea clave: GET consulta, POST crea, PUT reemplaza, PATCH ajusta, DELETE destruye. El servidor decide cómo implementarlo, pero esas son las convenciones estándar del ecosistema.*

Códigos de retorno HTTP

Cada respuesta incluye un **status code** que indica qué pasó. Son como el tono emocional del servidor.

2xx → éxito

- **200 OK**: todo bien
- **201 Created**: se creó algo correctamente
- **204 No Content**: éxito pero sin contenido para devolver

3xx → redirecciones

- **301 Moved Permanently**: el recurso cambió de URL
- **302 Found**: redirección temporal

4xx → error del cliente

- **400 Bad Request**: request mal formado

- **401 Unauthorized**: falta autenticación
- **403 Forbidden**: autenticado pero sin permiso
- **404 Not Found**: recurso inexistente

5xx → error del servidor

- **500 Internal Server Error**: algo explotó internamente
- **502 Bad Gateway**: fallo en comunicación entre servidores
- **503 Service Unavailable**: servidor caído o saturado

Los códigos no son decoración: permiten automatizar decisiones del lado cliente.

Idempotencia

Un método es **idempotente** cuando repetir la misma request varias veces produce **el mismo resultado final**.

Ejemplo mental:

- GET /users → pedir lista 10 veces no cambia nada → idempotente ✔️
- DELETE /user/5 → borrarlo una vez o diez deja el mismo estado (no existe) → idempotente ✔️
- PUT /user/5 con mismos datos → siempre termina igual → idempotente ✔️
-

POST /users → cada vez crea uno nuevo → no idempotente ❌

Esto importa muchísimo para:

- reintentos automáticos
- redes inestables
- diseño de APIs robustas

La idempotencia es básicamente inmunidad contra el botón “refresh del pánico”.

Autenticación de requests

Sirve para verificar **quién hace la request**.

Métodos comunes:

API Key

Se envía una clave en header o query. Simple pero menos seguro si se expone.

Basic Auth

Usuario y contraseña codificados en Base64 en el header Authorization. Hoy se usa poco sin HTTPS.

Bearer Token (JWT, OAuth, etc.)

Se envía:

Authorization: Bearer TOKEN

Es el estándar moderno. El token contiene identidad y permisos firmados.

Session cookies

El servidor guarda sesión y el navegador envía cookie automáticamente. Muy usado en apps web tradicionales.

Idea conceptual:

- Autenticación = quién sos

- Autorización = qué podés hacer

Muchos sistemas confunden ambos, y ahí nacen los bugs legendarios.

APIs REST

Las APIs REST son básicamente una forma ordenada de exponer datos y acciones usando HTTP, pero pensadas como si el sistema fuera un **universo de recursos**. Eso hace que el diseño sea predecible, cacheable y entendible incluso para humanos con café insuficiente.

Modelo de recursos en una API REST

En REST, un **recurso** es cualquier entidad del sistema:

- usuarios
- artículos
- comentarios
- pedidos
- productos

Cada recurso:

- tiene **una URL única**
- se manipula con **métodos HTTP**
-

se representa normalmente en **JSON**

Ejemplo :

```
/users        -> colecci
/users/15     -> usuario
/articles
/articles/9
/comments
/comments/33
```

Las URLs representan **cosas**, no acciones.

❌ /getUsers

❌ /createArticle

✓ /users

✓ /articles

La acción la define el método HTTP, no el nombre de la ruta.

Cuándo usar cada método HTTP en REST

GET → obtener datos

No cambia el estado del servidor.

```
GET /users
GET /users/15
```

```
GET /articles/3/comments
```

POST → crear recurso nuevo

Se usa sobre la **colección**.

```
POST /users
POST /articles
POST /articles/3/comment
```

Body ejemplo:

```
{
  "name": "Carolina",
  "email": "caro@email.c
}
```

PUT → reemplazar recurso completo

Se usa sobre un recurso específico.

```
PUT /users/15
```

Debe enviar el objeto completo:

```
{
  "name": "Carolina Lien
  "email": "nuevo@email.
}
```

PATCH → modificar parcialmente

Solo envía los campos que cambian.

```
PATCH /users/15
```

```
{
  "email": "nuevo@email.
}
```

DELETE → eliminar recurso

```
DELETE /users/15
DELETE /comments/9
```

JSON como estándar de formatting

JSON (JavaScript Object Notation) es el formato dominante porque:

- es **liviano**
- es **legible por humanos**
- es **fácil de parsear**
-

funciona en casi cualquier lenguaje

Ejemplo de respuesta REST típica:

```
{
  "id": 15,
  "name": "Carolina",
  "email": "caro@email.c
  "created_at": "2026-02
}
```

Lista de recursos:

```
{
  "data": [
    { "id": 1, "title":
    { "id": 2, "title":
  ],
  "total": 2
}
```

JSON no es obligatorio en REST… pero hoy ignorarlo sería como abrir una pizzería sin harina.

Ejemplo de routing básico (usuarios, artículos, comentarios)

Imaginemos un blog con usuarios que escriben artículos y artículos con comentarios.

👤 Usuarios

```
GET    /users          -
GET    /users/{id}     -
POST   /users          -
PUT    /users/{id}     -
PATCH  /users/{id}     -
DELETE /users/{id}     -
```

📰 Artículos

```
GET    /articles
GET    /articles/{id}
POST   /articles
PUT    /articles/{id}
PATCH  /articles/{id}
DELETE /articles/{id}
```

💬 Comentarios (recurso anidado)

Comentarios pertenecen a artículos:

```
GET    /articles/{id}/co
POST   /articles/{id}/co
GET    /comments/{id}
PATCH  /comments/{id}
DELETE /comments/{id}
```

Esto refleja relaciones reales:

- un artículo tiene comentarios
- cada comentario tiene ID propio

Se evita:

```
/createCommentForArticle
/getArticleComments
```

Porque REST modela **entidades**, no procedimientos.

Webhooks

Los **webhooks** son mecanismos que permiten a un sistema enviar notificaciones en tiempo real a otro cuando ocurre un evento específico, eliminando la necesidad de consultas constantes (polling). Por ejemplo, un webhook puede notificar a un sistema de inventario cuando se registra una nueva orden médica.

> ***Ejemplo en healthcare:*** *Un sistema de gestión de pacientes expone una API REST para actualizar historiales médicos, mientras que un webhook notifica al sistema de alertas cuando se registra una reacción adversa.*

Un **webhook** es, en esencia, un **endpoint HTTP público** al que otro sistema le hace un request automático cuando ocurre un evento.
Si una API tradicional es “yo pregunto si pasó algo”, un webhook es “te aviso apenas pase”. Es el modo notificación push del universo backend.

Webhook = endpoint HTTP que recibe requests

Para usar un webhook:

1. Tu sistema expone una URL accesible por internet

2. Esa URL acepta requests HTTP (normalmente POST)

3. Otro servicio envía datos allí cuando ocurre un evento

Ejemplo ficticio:

```
https://mi.servicio/webh
```

Ese endpoint puede recibir algo como:

```
POST https://mi.servicio/
Content-Type: application
```

```
{
  "event": "order.created
  "order_id": 8452,
  "total": 120.50,
  "currency": "USD"
}
```

Cuando llega esa request, tu backend:

- valida que venga del proveedor correcto
- procesa los datos
- guarda info
- dispara acciones internas

Es literalmente un **request entrante automatizado**.

Ejemplo

Imaginemos:

- tienes una tienda online
- usas un servicio de pagos externo
- quieres saber cuándo un pago fue confirmado

El servicio de pagos permite configurar:

```
Webhook URL: https://mi.
```

Cuando el cliente paga, el proveedor envía:

```
{
  "event": "payment.succ
  "payment_id": "pay_123
  "order_id": 8452,
  "status": "approved"
}
```

Tu servidor recibe ese POST y:

- marca la orden como pagada
- envía email al cliente
- genera factura

Todo automático. Sin polling. Sin preguntar cada 5 segundos “¿ya pagó? ¿ya pagó? ¿ya pagó?”.

Los webhooks eliminan ese comportamiento ansioso típico de sistemas novatos.

Buenas prácticas con webhooks

Un webhook serio suele:

- usar **POST**
- enviar **JSON**
- incluir **firma o token secreto** para verificar autenticidad
- esperar respuesta **200 OK** rápida (si no, reintenta)

Tu endpoint debería:

1. verificar firma/autenticación

2.

responder rápido

3.

procesar en background si es pesado

Porque muchos proveedores reintentan si no reciben 200, y podrías procesar el mismo evento varias veces.

Fallbacks y Políticas de Espera

Los **fallbacks** son estrategias diseñadas para manejar errores o indisponibilidad de servicios externos, asegurando que el sistema continúe operando de forma degradada o con alternativas.

Las **políticas de espera** (`timeouts` , `retries` , `circuit breakers`) controlan cómo y cuándo se reintentan las solicitudes o se detiene la comunicación con un servicio problemático para evitar sobrecarga.

> ***Ejemplo:*** *Si la API de un laboratorio externo no responde, el sistema puede usar un fallback que consulte datos en caché o notifique al usuario para intervención manual.*

Ejemplos

Exponential backoff (espera exponencial)

Consiste en reintentar aumentando el tiempo entre intentos:

```
1s → 2s → 4s → 8s → 16s
```

Cuándo usarlo

- APIs externas saturadas

- errores 5xx temporales
- rate limits
- servicios cloud con throttling

Ejemplo realista:

Tu sistema llama a un proveedor de pagos y recibe **503 Service Unavailable**.

Reintentar inmediatamente solo empeora la congestión. El backoff exponencial:

- reduce presión sobre el servicio
- aumenta probabilidad de éxito
- evita loops agresivos

Este es el **estándar por defecto** para redes inestables o dependencias externas.

Random backoff (espera aleatoria)

Introduce un retraso **aleatorio** antes de reintentar.

```
retry in random(1s–5s)
```

Cuándo usarlo

- múltiples clientes reintentando a la vez
- sistemas distribuidos
- workers paralelos
- colas de procesamiento masivo

Ejemplo:
Miles de instancias intentan procesar un job y todas fallan por timeout.
Si todas reintentan a los 2 segundos exactos → pico brutal simultáneo.

Con random delay:

- los reintentos se dispersan
- se evita el **thundering herd problem** (manada estampida digital)

Random es el antídoto contra la sincronización accidental.

Exponential backoff + jitter (mezcla ideal)

La práctica moderna es:

```
espera exponencial + com
```

Ejemplo:

```
base = 2^n segundos
delay = random(0, base)
```

Cuándo usarlo

- microservicios
- cloud infra
- SDKs oficiales (AWS, Google, Stripe, etc.)
- webhooks retry
- APIs públicas

Es la **mejor opción general** porque:

- reduce carga progresivamente

- evita sincronización masiva
- mejora resiliencia

Muchos sistemas grandes lo usan como política estándar.

Fallback a cache

En vez de reintentar, devuelves datos guardados.

Cuándo usarlo

- contenido relativamente estable
- consultas de catálogo
- perfiles de usuario
- precios no críticos en tiempo real

Ejemplo:

- servicio de recomendaciones falla
- devuelves última recomendación cacheada

Usuario obtiene algo útil en vez de error.

Esto prioriza **disponibilidad sobre frescura**.

Fallback a respuesta por defecto

Se devuelve un valor fijo o simplificado.

Cuándo usarlo

- features secundarias
- personalización opcional
- métricas
- datos no críticos

Ejemplo:

- sistema de scoring falla
- mostrar ranking genérico

El usuario ni nota el problema.

Esto prioriza **experiencia sobre precisión**.

Circuit breaker (no reintentar temporalmente)

Cuando detectas que un servicio está fallando repetidamente:

- cortas llamadas por un tiempo
-

respondes fallback inmediato

- pruebas luego de enfriar

Cuándo usarlo

- dependencias críticas inestables
- APIs externas lentas
- evitar cascada de fallos

Ejemplo:

- API externa tarda 20s y falla
- sin breaker: saturas threads y caes tú también
- con breaker: respondes rápido con fallback

Esto protege **tu sistema primero**.

Regla mental rápida

- Error temporal remoto → **exponential + jitter**
- Muchos clientes simultáneos → **random o jitter**

- Datos no críticos → **cache fallback**
- Feature secundaria → **default response**
- Servicio inestable → **circuit breaker**

Un sistema robusto no intenta ser perfecto: intenta **fallar elegantemente**.

La arquitectura moderna no trata de evitar fallos (eso es fantasía), sino de diseñar cómo sobrevivirlos sin que el usuario vea humo saliendo del servidor. Si querés, se puede seguir con ejemplos de código reales (Python, Node o pseudo-backend) para implementar cada estrategia y ver cómo se combinan en producción.

Logs, Métricas y Observabilidad

La **observabilidad** es la capacidad de entender el estado interno de un sistema a partir de sus salidas externas, fundamental para sistemas críticos en healthcare.

- **Logs:** registros detallados de eventos y errores.
- **Métricas:** datos cuantitativos como latencia, tasa de error, throughput.
- **Alertas:** notificaciones automáticas basadas en métricas para detectar anomalías.

> ***Caso práctico:*** *Un sistema de monitoreo detecta un aumento en la latencia de una API crítica y genera una alerta para el equipo de soporte.*

DEBUG → detalle técnico para desarrolladores

Se usa para información **muy específica**, útil al depurar.

Ejemplos:

- parámetros recibidos por una función
- query SQL generada

- respuesta cruda de una API externa
- flujo paso a paso de un algoritmo

Caso:

```
DEBUG: Payload recibido (
DEBUG: Conectando a DB h(
```

En producción suele desactivarse porque:

- genera mucho volumen
- puede incluir datos sensibles
- afecta rendimiento

DEBUG = “modo microscopio”.

INFO → eventos normales importantes

Registra lo que **debería pasar normalmente**.

Ejemplos:

- servidor iniciado
- usuario autenticado
- job procesado correctamente
- webhook recibido
- archivo generado

Caso:

```
INFO: Usuario 15 inició s
INFO: Orden 8452 creada c
INFO: Worker de emails in
```

INFO cuenta la historia normal del sistema.

Si lees solo INFO, deberías entender qué está pasando sin ver DEBUG.

WARNING → algo raro pero no fatal

Indica una situación inesperada o potencial problema, pero el sistema sigue funcionando.

Ejemplos:

- API externa lenta
- intento de login fallido
- uso de feature deprecada
- retry por timeout
- configuración incompleta

Caso:

```
WARNING: Timeout llamand
WARNING: Usuario intentó
```

WARNING es como el check engine del auto: todavía anda, pero ojo.

ERROR → fallo que afecta una operación

La app no pudo completar algo importante.

Ejemplos:

-

excepción no controlada en request

- no se pudo guardar en DB
- API externa devuelve error crítico
- fallo al procesar pago

Caso:

```
ERROR: No se pudo crear
ERROR: Fallo procesando
```

ERROR implica que **algo falló y requiere atención**.

CRITICAL / FATAL → sistema comprometido

Problemas graves que pueden detener la app.

Ejemplos:

- base de datos inaccesible
- configuración inválida al iniciar

- falta de variables críticas
- servicio principal caído

Caso:

```
CRITICAL: No se pudo ini
CRITICAL: Memoria insufi
```

CRITICAL significa: “esto no se arregla solo”.

Sentry (monitoreo de errores)

Herramientas tipo **Sentry** detectan errores automáticamente y notifican cuando la app falla en producción.

Qué hacen:

- capturan excepciones no manejadas
- guardan stack trace completo
- registran contexto (usuario, request, variables)
- agrupan errores repetidos

- envían alertas (email, Slack, etc.)

En vez de leer logs manualmente, Sentry te dice:

> *“Hey, tu endpoint /payments está rompiéndose desde hace 3 minutos.”*

Ejemplo práctico

Un usuario intenta pagar y ocurre:

```
raise ValueError("Monto
```

Sentry captura:

- tipo de error
- línea exacta
- endpoint
- payload
- usuario afectado

- frecuencia del error

Te muestra algo como:

```
ValueError: Monto inváli
POST /payments
Usuarios afectados: 27
Primera vez: 10:03
Última vez: 10:07
```

Esto reduce el clásico:

> *“Un cliente dice que algo no funciona pero no sabemos qué”.*

Cómo conviven logs y Sentry

- **DEBUG/INFO/WARNING** → observabilidad y trazabilidad
- **ERROR/CRITICAL** → problemas reales
- **Sentry** → detector automático de fallos importantes

Logs = diario del sistema

Sentry = alarma contra incendios

Uno cuenta la historia. El otro grita cuando algo se prende fuego.

[[explicar sistemas como Sentry, que notifican cuando la app tiene un error]]

Conexión con conceptos avanzados

Aunque esta unidad introduce estos conceptos, en módulos posteriores profundizaremos en patrones como RAG, multimodalidad y workflows con intervención humana, que se apoyan en una base sólida de APIs, fallbacks y observabilidad.*

Aplicación Práctica en Healthcare y Biotecnología

En entornos sensibles como hospitales o laboratorios, la automatización debe garantizar alta disponibilidad y trazabilidad. Por ejemplo, un pipeline automatizado que procesa resultados de pruebas genéticas debe integrar APIs de bases de datos, manejar fallos en servicios externos y registrar cada paso para auditoría y diagnóstico.

La correcta implementación de fallbacks evita interrupciones que podrían afectar decisiones clínicas. La observabilidad permite detectar problemas antes de que impacten al paciente, facilitando intervenciones rápidas.

Este enfoque no solo mejora la confiabilidad sino que también cumple con normativas de calidad y seguridad propias del sector.

> ***Preparación para práctica:*** *En la siguiente sección, aplicarás estos conceptos para diseñar un diagrama de arquitectura y definir estrategias de fallback para un caso realista.*

Para practicar

Instrucciones:

Instrucciones para la Práctica

1.

Diagramar arquitectura API-centric:

- Diseña un diagrama que integre al menos tres servicios mediante APIs REST y webhooks.
- Incluye puntos de orquestación y comunicación en tiempo real.

2.

Definir estrategias de fallback:

- Para cada integración, especifica qué fallback implementarías en caso de fallo o latencia excesiva.
- Considera políticas de espera y degradación controlada.

3.

Incluir elementos de observabilidad:

- Indica qué logs y métricas serían críticos para monitorear el flujo.
- Propón alertas basadas en métricas clave.

4. **Contextualiza tu diseño en un escenario de healthcare o biotecnología:**

- Explica brevemente cómo tu diseño mejora la robustez y confiabilidad del sistema.

Consejo: Usa herramientas de diagramación como draw.io, Lucidchart o incluso papel y lápiz para crear tu diagrama. Sé claro y conciso en las definiciones de fallback y observabilidad.

Reflexión: ¿Cómo tu diseño anticipa y mitiga riesgos operativos en producción?

Entregable:

Documento Google Docs de 4-5 páginas que incluya:

- Portada con título y contexto del caso de uso.
- Diagrama detallado de la arquitectura API-centric con webhooks.
- Descripción de estrategias de fallback y políticas de espera para cada integración.
- Plan de logging, métricas y alertas para monitoreo y diagnóstico.
- Análisis breve de cómo estas prácticas aseguran la continuidad y calidad del servicio en un entorno healthcare o biotecnología.

El documento debe ser profesional, con diagramas claros y explicaciones técnicas precisas, orientadas a un público técnico avanzado.